Program Studi Teknik Informatika
Institut Teknologi Sumatera

MODUL PRAKTIKUM

Sistem Operasi

IF25-12007 | Genap 2025/2026

Mata Kuliah	Sistem Operasi (IF25-12007)
Semester	Genap 2025/2026
Institusi	Institut Teknologi Sumatera

© 2026 — Institut Teknologi Sumatera

TIM PENGAJAR & ASISTEN

Dosen Pengampu

Martin Clinton Tosima Manullang, S.T., M.T., Ph.D.

Ilham Firman Ashari, S.Kom., M.T

I Wayan Wiprayoga Wisesa, S.Kom., M.Kom.

Eko Dwi Nugroho, S.Kom., M.Cs.

Asisten Praktikum

Ahmat Prayoga Sembiring	123140053
Andryano Shevchenko Limbong	123140205
Aziz Kurniawan	122140097
Dito Rifki Irawan	122140153
Ebentua Philippus Limbong	123140086
Elsa Elisa Yohana Sianturi	122140135
Fathan Andi Kartagama	122140055
Miftahul Khoiriyah	123140064
Muhammad Bintang Al-Fasya	123140098
Muhammad Fadhil Zurani	122140146
Reza Chairul Manam	120140086
Stevanus Cahya Anggara	123140038
Varasina Farmadani	123140107
Zakhi Algifari	122140198

Contents

1 Analisis Race Condition 5

1.1 Tujuan dan Output Praktikum . . . 5

1.1.1 Tujuan Praktikum . . . 5

1.1.2 Output Praktikum . . . 5

1.2 Dasar Teori . . . 6

1.2.1 Race Condition . . . 6

1.2.2 Concurrency dan Shared Memory . . . 7

1.2.3 Critical Section dan Mutual Exclusion . . . 8

1.2.4 Preemption dan Cooperative Scheduling . . . 8

1.2.5 Producer-Consumer . . . 8

1.3 Alat dan Bahan . . . 10

1.3.1 Perangkat Keras . . . 10

1.3.2 Perangkat Lunak (Bahan Praktikum) . . . 10

1.4 Langkah-Langkah Praktikum . . . 11

1.4.1 Persiapan Lingkungan . . . 11

1.4.2 Sesi 1: Menjalankan Demo Race Condition . . . 12

2 Implementasi Mutex dan Locks 14

2.1 Tujuan dan Output Praktikum . . . 14

2.1.1 Tujuan Praktikum . . . 14

2.1.2 Output Praktikum . . . 14

2.2 Dasar Teori . . . 16

2.2.1 Konsep Semaphore . . . 16

2.2.2 Mutex (Mutual Exclusion Lock) . . . 17

2.2.3 Pendekatan Logika Sinkronisasi Producer-Consumer . . . 18

2.3 Alat dan Bahan . . . 20

2.3.1 Perangkat Keras 20
2.3.2 Perangkat Lunak (Bahan Praktikum) 20
2.4 Langkah-Langkah Praktikum. 21
2.4.1 Persiapan Lingkungan 21
2.4.2 Sesi 1: Mengimplementasikan Logika Semaphore 21
2.4.3 Sesi 2: Mengimplementasikan Mutex Lock 23
2.4.4 Sesi 3: Integrasi Sinkronisasi pada Simulasi Kernel 23
2.4.5 Sesi 4: Kompilasi dan Verifikasi Sistem OS 25

Bibliography **27**

CHAPTER 1

Analisis Race Condition

1.1 Tujuan dan Output Praktikum

1.1.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami konsep *Race Condition* pada sistem operasi *multithreading*.
2. Mengidentifikasi *Critical Section* pada penggunaan memori bersama (*shared memory*).
3. Menganalisis dampak ketiadaan mekanisme sinkronisasi terhadap integritas data pada *cooperative scheduling*.
4. Menghubungkan perilaku `thread_yield()` dengan konsep preemption pada sistem nyata.

1.1.2 Output Praktikum

Pada akhir praktikum ini, mahasiswa diharapkan menghasilkan:

- Eksekusi kernel yang mendemonstrasikan anomali *Producer-Consumer problem* tanpa sinkronisasi.
- Laporan analisis log yang menunjukkan terjadinya *data inconsistency* akibat *race condition*.

1.2 Dasar Teori

1.2.1 Race Condition

Race Condition (Kondisi Balapan) merupakan suatu anomali yang terjadi ketika dua atau lebih *thread* secara bersamaan mengakses dan memodifikasi sumber daya yang sama (*shared resources*). Hal ini mengakibatkan ketidakkonsistenan pada hasil akhir, bergantung pada urutan *thread* mana yang dieksekusi lebih dahulu oleh sistem [1].

Sebagai analogi, bayangkan ada satu buah mobil dan dua orang pengguna. Pengguna pertama ingin mengendarai mobil tersebut, sementara pengguna kedua ingin mengecatnya menjadi warna putih. Jika kedua tindakan ini dilakukan secara bersamaan tanpa adanya koordinasi, hasil akhirnya akan menjadi rancu. Bisa jadi mobil tersebut dibawa pergi saat baru dicat sebagian, atau justru pengecatan gagal dilakukan karena mobilnya sudah keburu dibawa pergi.

Contoh *race condition* pada bahasa tingkat tinggi seperti C++ (standar C++23 digunakan untuk kemudahan pengenalan konsep):

```cpp
#include <print>
#include <thread>

// Variabel counter sebagai shared resources
auto counter = 0;

// Fungsi increment melakukan penambahan pada variable counter sampai 100.000 kali
auto increment() -> void {
    for (auto i = 0; i < 100'000; ++i) {
        counter++;
    }
}

auto main() -> int {
    // Dua thread berbeda sama-sama menjalankan fungsi increment
```

```
    yang sama
    std::thread t1(increment);
    std::thread t2(increment);

    // Menunggu thread t1 dan t2 selesai
    t1.join();
    t2.join();

    // Hasil akhir akan acak dan tidak konsisten
    std::println("{}", counter);
}
```

Kode 1.1: Contoh Race Condition pada C++

Pada materi ini diperkenalkan beberapa konsep teknis, yaitu:

1. Concurrency dan Shared Memory
2. Critical Section dan Mutual Exclusion
3. Producer-Consumer (Bounded-Buffer Problem)

1.2.2 Concurrency dan Shared Memory

Concurrency adalah kemampuan sistem operasi untuk menjalankan beberapa proses atau *thread* seolah-olah secara bersamaan. Tujuannya adalah untuk meningkatkan efisiensi penggunaan prosesor. Melalui mekanisme perpindahan konteks (*context switching*) pada satu inti CPU, sistem dapat mengerjakan beberapa tugas sekaligus secara bergantian dengan sangat cepat [2]. Meskipun memberikan kinerja yang lebih baik, *concurrency* memunculkan sejumlah tantangan baru, seperti *Race Condition*, *Deadlock*, dan *Blocking*.

Sementara itu, *Shared Memory* merupakan area memori yang sengaja dialokasikan agar dapat diakses secara bersama-sama oleh dua atau lebih proses maupun *thread* yang berbeda.

1.2.3 Critical Section dan Mutual Exclusion

Critical Section adalah bagian dari kode program yang mengakses *shared resource* dan **hanya boleh dieksekusi oleh satu thread pada satu waktu**. Jika lebih dari satu thread memasuki *critical section* secara bersamaan, *race condition* dapat terjadi dan integritas data tidak dapat dijamin.

Terdapat tiga syarat yang harus dipenuhi oleh mekanisme proteksi *critical section*:

1. **Mutual Exclusion** — Hanya satu *thread* yang boleh berada di dalam *critical section* pada satu waktu.
2. **Progress** — Jika tidak ada *thread* di dalam *critical section*, *thread* yang menunggu harus dapat segera masuk tanpa penundaan.
3. **Bounded Waiting** — Setiap *thread* yang menunggu harus dijamin akan mendapat giliran dalam waktu yang terbatas (tidak *starving*).

1.2.4 Preemption dan Cooperative Scheduling

Preemption adalah tindakan sistem operasi untuk menghentikan sementara (interupsi) eksekusi proses atau *thread* yang sedang berjalan agar CPU dapat dialokasikan ke *thread* lain. Pada sistem operasi modern yang bersifat *preemptive scheduling*, perpindahan ini dapat terjadi kapan saja secara paksa tanpa persetujuan dari *thread* tersebut (biasanya dikendalikan oleh *timer interrupt* dari perangkat keras). Sebaliknya, pada *cooperative scheduling*, sebuah *thread* harus menyerahkan kontrol CPU kepada *scheduler*.

1.2.5 Producer-Consumer

Producer-Consumer problem (atau *Bounded-Buffer problem*) adalah skenario klasik dalam sistem operasi yang melibatkan dua jenis thread:

- **Producer** — *thread* yang menghasilkan data dan menulisnya ke *shared buffer*.
- **Consumer** — *thread* yang mengonsumsi data dengan membacanya dari *shared buffer*.

Masalah muncul ketika Producer dan Consumer mengakses *shared buffer* tanpa koordinasi:

- Producer bisa menimpa data yang belum sempat dibaca Consumer (*data loss*).
- Consumer bisa membaca data lama atau data yang sudah diperbarui di tengah jalan (*stale read*).
- Log yang dicetak ke layar bisa terpotong atau bercampur (*interleaved output*).

Tabel 1.1 merangkum hal-hal penting terkait teori di atas.

Tabel 1.1: Tabel Ringkasan Dasar Teori Modul 9

Konsep	Penjelasan Singkat
Race Condition	Kondisi di mana dua atau lebih *thread* mengakses *shared resource* secara bersamaan sehingga hasil akhir tidak konsisten.
Critical Section	Bagian kode yang mengakses *shared resource* dan hanya boleh dijalankan oleh satu *thread* pada satu waktu.
Shared Memory	Variabel atau area memori yang dapat diakses oleh lebih dari satu *thread* (`shared_buffer`).
Preemption	Kondisi di mana CPU berpindah ke *thread* lain di tengah eksekusi; disimulasikan oleh `thread_yield()`.
Producer-Consumer	Pola dua *thread* yang saling berbagi *buffer*: *Producer* menulis, *Consumer* membaca, tanpa koordinasi hasilnya tidak konsisten.

1.3 Alat dan Bahan

1.3.1 Perangkat Keras

- PC atau Laptop dengan arsitektur prosesor basis *host* x86/x64 yang mendukung kelanjutan kompilasi emulator.
- Memori minimal 4GB RAM ruang eksekusi.

1.3.2 Perangkat Lunak (Bahan Praktikum)

- OS *Host* utama (Windows dengan WSL/GitBash, Linux, atau macOS).
- Text Editor (misalnya Visual Studio Code) untuk menyunting berkas kode bahasa C dan *Assembly*.
- Instalasi Docker Engine (`docker.io`) untuk menyalakan lingkungan isolasi kompilasi.
- Emulator Hardware x86: `qemu-system-x86` (atau instalasi QEMU setara) yang akan mengeksekusi binari hasil OS.
- Direktori *Starter Code* OS 32-bit yang sudah disiapkan sebelumnya (berisi direktori `src` dengan kerangka lengkap `kernel.c`, `thread.h`, `thread.c`, `switch.asm`, `scheduler.h`, dsb).

1.4 Langkah-Langkah Praktikum

1.4.1 Persiapan Lingkungan

Sebelum memulai praktikum, pastikan *toolchain* dan *source code* OS sudah tersedia. Karena praktikum ini merupakan lanjutan dari pembuatan penjadwal (Minggu 4), implementasi `thread_create()`, `thread_yield()`, dan `next_thread_to_run()` di `thread.c` **harus sudah selesai dan berjalan dengan benar**. Jika Anda menggunakan *starter code* mentah, simulasi akan terhenti seketika dan menghasilkan pesan *error*: `kernel_main() kembali dari thread_block()!`.

1. Buat direktori kerja baru khusus untuk praktikum minggu ke-5, lalu pindah ke direktori tersebut.

```
mkdir praktikum-minggu5
cd praktikum-minggu5

```

Kode 1.2: Membuat direktori kerja baru

2. Lakukan *clone* ulang *source code* OS dari repositori GitHub untuk memastikan Anda memulai dengan lingkungan (*environment*) yang bersih.

```
git clone https://github.com/sinavarasina/starter-code-os32.
    git
cd starter-code-os32

```

Kode 1.3: Mengunduh starter code melalui Git

3. **Langkah Kritis:** Timpa (*replace*) file `src/thread.c` bawaan repositori dengan implementasi `thread.c` yang sudah berfungsi dari Minggu 4. Anda dapat menggunakan perintah `wget` berikut untuk langsung mengunduh kode yang sudah dipersiapkan oleh asisten ke dalam folder `src/`.

```
# Mengunduh file thread.c (versi lengkap) dan menimpanya ke folder src/
wget -O src/thread.c https://pastebin.com/raw/mb4DJf1p
# Jika wget belum terinstal, buka tautan pada peramban web dan salin kontennya secara manual

```

Kode 1.4: Mengganti thread.c dengan versi yang sudah lengkap

4. Verifikasi bahwa seluruh berkas sumber (*source code*) sudah lengkap.

```
ls src/
# Harus menampilkan: boot.asm  kernel.c  synch.c  synch.h
#                    switch.asm  thread.c  thread.h
#                    types.h  vga.c  vga.h

```

Kode 1.5: Verifikasi direktori source code

1.4.2 Sesi 1: Menjalankan Demo Race Condition

Pada sesi ini, praktikan akan mengompilasi dan menjalankan OS dengan *flag* `SYNCHRONIZATION`. **Tidak perlu mengubah kode apa pun**, cukup amati keluaran (*output*) yang muncul di layar QEMU.

Perhatikan bahwa pada file `kernel.c`, terdapat dua pemanggilan `thread_yield()` yang sengaja ditempatkan sebagai "jebakan" di tengah area *critical section*:

- Di dalam `producer()`: setelah `shared_buffer = item` tetapi **sebelum** mencetak log.
- Di dalam `consumer()`: setelah mencetak teks `"[Consumer] membaca item "` tetapi **sebelum** mencetak angkanya.

1. *Aktivasi lingkungan tersetup & terisolasi.*

```
    ./run-docker.sh # Opsional bagi pengguna Linux, namun disarankan menggunakan Docker demi kemudahan
```

```
2
```

Kode 1.6: Menjalankan Docker menggunakan script

2. *Compile* OS dengan *flag* `SYNCHRONIZATION` diaktifkan.

```
1 make DEMO=-DSYNCHRONIZATION
2
```

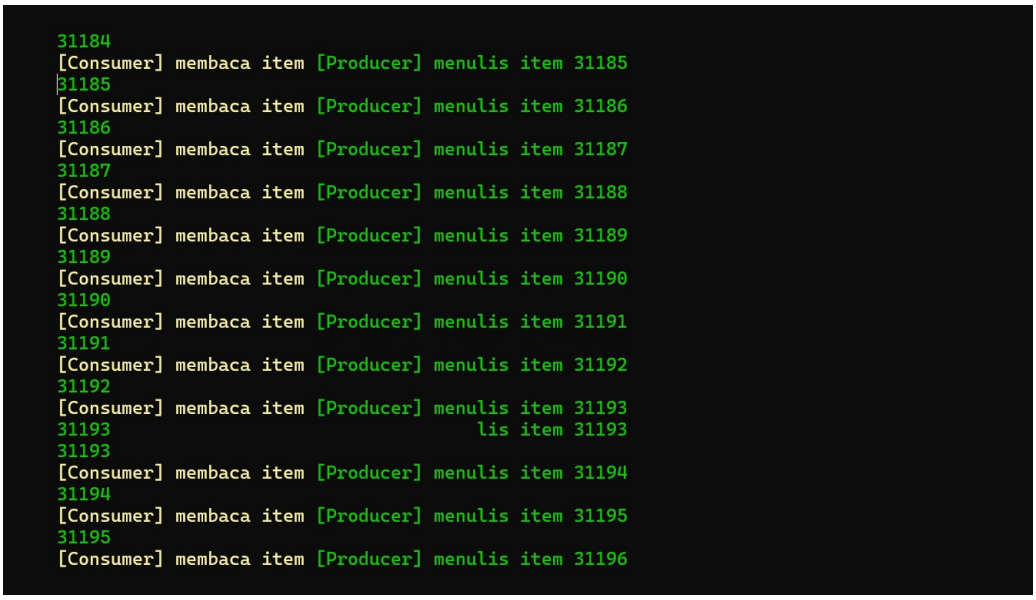
Kode 1.7: Compile dengan flag SYNCHRONIZATION

3. Jalankan kernel di QEMU dan amati keluaran (*output*) di layar.

```
1 make run
2
```

Kode 1.8: Menjalankan kernel di QEMU

4. Amati keluaran yang muncul selama kurang lebih 10-15 detik, ambil tangkapan layar (*screenshot*), lalu tutup QEMU. Lihat anomali yang ditemukan: apakah ada baris yang terpotong, angka yang tidak urut, atau keluaran yang bercampur?



Gambar 1.1: Contoh keluaran (*output*) QEMU yang menunjukkan *race condition* pada Producer-Consumer tanpa sinkronisasi. Perhatikan baris yang terpotong dan angka yang tidak konsisten.

CHAPTER 2

Implementasi Mutex dan Locks

2.1 Tujuan dan Output Praktikum

2.1.1 Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

1. Memahami cara kerja *semaphore* sebagai mekanisme dasar sinkronisasi antar *thread*.
2. Memahami konsep *mutex* (*Mutual Exclusion Lock*) dan perbedaannya dengan *semaphore* secara semantik maupun struktural.
3. Mengimplementasikan mekanisme penahanan (*wait*) dan pelepasan (*signal*) sumber daya, serta sistem penguncian eksklusif pada tingkat kernel.
4. Memverifikasi efektivitas mekanisme sinkronisasi yang dibangun dalam mencegah dan menghilangkan *race condition* yang diamati pada modul sebelumnya.

2.1.2 Output Praktikum

Pada akhir praktikum ini, mahasiswa diharapkan menghasilkan:

- Implementasi algoritma sinkronisasi kernel yang valid dan mampu meregulasi antrean *thread* dengan tepat.
- Simulasi sistem operasi yang mendemonstrasikan penyelesaian skenario *Producer-Consumer* dengan eksekusi yang tersinkronisasi, rapi, dan konsis-

ten.

- Laporan analisis yang menjabarkan bagaimana struktur *semaphore* dan *lock* bekerja sama secara harmonis untuk melindungi area *critical section* dari interupsi paksa.

2.2 Dasar Teori

2.2.1 Konsep Semaphore

Semaphore adalah sebuah struktur data abstrak yang digunakan untuk mengoordinasikan akses beberapa *thread* atau proses terhadap sumber daya (*shared resource*) yang jumlahnya terbatas. Konsep ini pertama kali diperkenalkan oleh ilmuwan komputer Edsger W. Dijkstra pada tahun 1965.

Secara konseptual, *semaphore* bekerja seperti "keranjang berisi token". Jika sebuah *thread* ingin melakukan pekerjaan, ia harus mengambil satu token dari keranjang. Jika keranjang kosong, *thread* tersebut harus menunggu (mengantre) sampai ada token yang dikembalikan oleh *thread* lain.

Di dalam sistem operasi, sebuah *semaphore* terdiri dari dua komponen utama:

1. Sebuah variabel bilangan bulat (integer) non-negatif yang mewakili "jumlah token" atau kapasitas sumber daya.
2. Sebuah antrean (*waiting queue*) untuk menampung *thread* yang sedang menunggu giliran.

Semaphore dikendalikan melalui dua operasi atomik (tidak dapat diinterupsi):

- **Operasi `Wait(S)` (P atau *Proberen*)**: Mengurangi nilai variabel semaphore. Jika nilainya menjadi 0 (atau kurang), maka *thread* yang mengeksekusi operasi ini akan diubah statusnya menjadi *Blocked* / *Asleep* dan dimasukkan ke dalam antrean. Pada sistem OS, operasi ini dinamakan `sema_down()`.
- **Operasi `Signal(S)` (V atau *Verhogen*)**: Menambah nilai variabel semaphore. Jika ada *thread* yang sedang mengantre (tertidur), operasi ini akan membangunkan (*Wake up*) salah satu *thread* tersebut agar siap dieksekusi kembali oleh CPU. Pada sistem OS, operasi ini dinamakan `sema_up()`.

Berdasarkan kapasitasnya, semaphore dibagi menjadi dua:

1. **Counting Semaphore**: Nilainya bisa berapa saja (≥ 0). Sangat berguna untuk mengontrol akses ke *resource* yang memiliki batasan jumlah, seperti jumlah tempat parkir kosong, atau sisa kapasitas *buffer*.
2. **Binary Semaphore**: Nilainya hanya dibatasi pada angka 0 dan 1. Sering digunakan untuk menjamin *Mutual Exclusion*.

2.2.2 Mutex (Mutual Exclusion Lock)

Mutex atau *Lock* adalah mekanisme sinkronisasi yang secara khusus dirancang untuk menjamin *Mutual Exclusion* pada area *Critical Section*.

Analogi paling mudah untuk memahami *Mutex* adalah mengibaratkannya seperti penggunaan bilik kamar mandi.

- Ketika seseorang (*Thread A*) memasuki bilik (*Critical Section*), ia akan mengunci pintu dari dalam (*Lock*).
- Orang lain (*Thread B*) yang datang kemudian harus mengantre di luar hingga pintu tersebut dibuka (*Unlock*).
- Kunci pintu hanya dapat dibuka dari dalam oleh orang yang sama yang sebelumnya menguncinya (*Thread A*).

Konsep Ownership (Kepemilikan)

Hal ini merupakan perbedaan mendasar antara *Mutex* dan *Semaphore*. *Mutex* memiliki konsep **Ownership** (Kepemilikan). Sebuah *Mutex* akan mengingat identitas *thread* mana yang berhasil menguncinya. Sistem operasi yang baik akan menolak operasi *Unlock* jika fungsi tersebut dipanggil oleh *thread* yang bukan pemiliknya.

Di dalam level implementasi (seperti pada OS), Mutex sering kali dibangun di atas *Binary Semaphore* dengan nilai awal 1, namun ditambah dengan variabel `holder` untuk mencatat siapa pemegang kunci tersebut.

Tabel 2.1: Perbedaan Konseptual Semaphore dan Mutex

Aspek	Semaphore	Mutex (Lock)
Analogi	Keranjang berisi token bebas.	Kunci gembok eksklusif.
Konsep Nilai	Bilangan bulat ≥ 0 (tergantung kapasitas).	Hanya 0 (Terkunci) atau 1 (Bebas).
Kepemilikan (*Ownership*)	Tidak ada. *Thread A* bisa melakukan `Wait`, dan *Thread B* bisa melakukan `Signal`.	Ada. Hanya *thread* pemegang kunci yang berhak melakukan *Release/Unlock*.
Kegunaan Utama	Sinkronisasi urutan kejadian dan manajemen jumlah sumber daya.	Melindungi *Critical Section* dari modifikasi serentak.

2.2.3 Pendekatan Logika Sinkronisasi Producer-Consumer

Pada Modul 9, telah didemonstrasikan bahwa masalah *Producer-Consumer* (*Bounded-Buffer*) menyebabkan anomali ketika *buffer* diakses tanpa aturan. Untuk menyelesaikannya, dibutuhkan gabungan dari *Semaphore* dan *Mutex*.

Logika penyelesaiannya membutuhkan 3 instrumen:

1. **`mutex_lock`**: Untuk melindungi variabel *buffer* agar tidak ditulis/dibaca bersamaan.
2. **`sema_empty`**: *Counting Semaphore* untuk menghitung jumlah slot kosong di dalam *buffer*.
3. **`sema_full`**: *Counting Semaphore* untuk menghitung jumlah slot terisi di dalam *buffer*.

Alur Logika Producer:

- Sebelum memproduksi data, *Producer* harus melakukan `Wait(sema_empty)`. Jika kapasitas *buffer* sudah penuh, *thread* ini akan ditidurkan hingga ada slot yang kosong.
- Jika ada slot kosong, *Producer* akan mengamankan *buffer* dengan `Lock(mutex)`.
- Setelah data dimasukkan ke dalam *buffer*, *Producer* membuka kembali pengaman dengan `Unlock(mutex)`.
- Terakhir, *Producer* mengirimkan sinyal `Signal(sema_full)` untuk memberi tahu *Consumer* bahwa ada data baru yang siap dibaca.

Alur Logika Consumer:

- Sebelum membaca data, *Consumer* harus melakukan `Wait(sema_full)`. Jika *buffer* sedang kosong, *thread* ini akan ditidurkan hingga ada data baru.
- Jika ada data yang tersedia, *Consumer* akan mengamankan *buffer* dengan `Lock(mutex)`.
- Setelah data selesai dibaca, *Consumer* membuka kembali pengaman dengan `Unlock(mutex)`.
- Terakhir, *Consumer* mengirimkan sinyal `Signal(sema_empty)` untuk memberi tahu *Producer* bahwa ada slot kosong yang baru saja ditinggalkan.

Tabel 2.2: Tabel Ringkasan Dasar Teori Modul 10

Konsep	Penjelasan Singkat
Wait / P() / sema_down	Operasi menahan/menurunkan *resource*. Memblokir *thread* jika *resource* habis.
Signal / V() / sema_up	Operasi melepaskan/menambah *resource*. Membangunkan *thread* dari antrean.
Ownership	Konsep kepemilikan pada *Mutex* yang memastikan hanya sang pengunci yang boleh membuka kunci.
Thread Blocking	Keadaan di mana *thread* diberhentikan paksa oleh OS dan dikeluarkan dari penjadwalan CPU hingga *event* sinkronisasi terjadi.

2.3 Alat dan Bahan

2.3.1 Perangkat Keras

- PC atau Laptop dengan arsitektur prosesor basis *host* x86/x64 yang mendukung kelanjutan kompilasi emulator.
- Memori minimal 4GB RAM ruang eksekusi.

2.3.2 Perangkat Lunak (Bahan Praktikum)

- OS *Host* utama (Windows dengan WSL/GitBash, Linux, atau macOS).
- Text Editor (misalnya Visual Studio Code) untuk menyunting berkas kode bahasa C dan *Assembly*.
- Instalasi Docker Engine (`docker.io`) untuk menyalakan lingkungan isolasi kompilasi.
- Emulator Hardware x86: `qemu-system-x86` (atau instalasi QEMU setara) yang akan mengeksekusi binari hasil OS.
- Direktori *Starter Code* OS 32-bit yang sudah disiapkan sebelumnya (berisi direktori `src` dengan kerangka lengkap `kernel.c`, `thread.h`, `thread.c`, `switch.asm`, `scheduler.h`, dsb).

2.4 Langkah-Langkah Praktikum

2.4.1 Persiapan Lingkungan

Pada praktikum ini, Anda akan menuliskan implementasi fungsi sinkronisasi di dalam `src/synch.c` dan menghubungkannya dengan simulasi di `src/kernel.c`.

1. Buka terminal, pastikan Anda berada di dalam direktori *source code* OS.
2. Bersihkan hasil *build* sebelumnya agar kompilasi berikutnya menggunakan kode terbaru yang akan Anda tulis.

```
make clean

```

Kode 2.1: Bersihkan build sebelumnya

2.4.2 Sesi 1: Mengimplementasikan Logika Semaphore

Semaphore adalah fondasi utama dari mekanisme sinkronisasi di OS. Buka file `src/synch.c` menggunakan teks editor Anda dan cari blok fungsi `sema_down` dan `sema_up`.

1. **Implementasi `sema_down()` (Operasi Wait / P)**
   Fungsi ini bertugas menahan *thread* jika *resource* sedang kosong (nilai 0). Ubah fungsi `sema_down()` menjadi seperti berikut:

```
void sema_down(struct semaphore *sema) {
    /* 1. Selama nilai 0, masukkan thread ke antrean dan tidurkan */
    while (sema->value == 0) {
        sema->waiters[sema->waiter_count++] = thread_current();
        thread_block(); // Serahkan CPU ke thread lain
    }
```

```
    /* 2. Jika sudah lolos (nilai > 0), kurangi nilai \textit{resource} */
    sema->value--;
}

```

Kode 2.2: Kode implementasi sema_down()

2. **Implementasi `sema_up()` (Operasi Signal / V)**
   Fungsi ini bertugas menambah nilai *resource* dan membangunkan *thread* yang sedang antri (jika ada). Ubah fungsi `sema_up()` menjadi seperti berikut:

```
void sema_up(struct semaphore *sema) {
    /* 1. Kembalikan/tambah nilai \textit{resource} */
    sema->value++;

    /* 2. Jika ada thread yang menunggu, bangunkan yang pertama (FIFO) */
    if (sema->waiter_count > 0) {
        struct thread *t = sema->waiters[0];

        // Geser sisa antrean ke depan (menghapus index 0)
        for (uint32_t i = 0; i < sema->waiter_count - 1; i++) {
            sema->waiters[i] = sema->waiters[i + 1];
        }
        sema->waiter_count--;

        // Bangunkan thread agar masuk ke ready queue lagi
        thread_unblock(t);
    }
}

```

Kode 2.3: Kode implementasi sema_up()

2.4.3 Sesi 2: Mengimplementasikan Mutex Lock

Setelah *Semaphore* berfungsi, mekanisme ini akan dibungkus menjadi sebuah *Mutex Lock*. Masih di dalam file `src/synch.c`, cari bagian `lock_acquire` dan `lock_release`.

1. **Implementasi `lock_acquire()`**
   Gunakan `sema_down` untuk mengunci *binary semaphore* milik lock, lalu catat siapa pemiliknya.

```
void lock_acquire(struct lock *lock) {
    sema_down(&lock->semaphore);      // Tunggu & ambil kunci
    lock->holder = thread_current(); // Catat pemegang kunci saat ini
}

```

Kode 2.4: Kode implementasi lock_acquire()

2. **Implementasi `lock_release()`**
   Hapus kepemilikan lock, lalu gunakan `sema_up` untuk membebaskan kunci agar bisa dipakai *thread* lain.

```
void lock_release(struct lock *lock) {
    lock->holder = NULL;              // Hapus identitas pemegang kunci
    sema_up(&lock->semaphore);        // Lepas kunci
}

```

Kode 2.5: Kode implementasi lock_release()

3. **Simpan** file `synch.c`. Saat ini *backend* sinkronisasi Anda sudah selesai.

2.4.4 Sesi 3: Integrasi Sinkronisasi pada Simulasi Kernel

Mekanisme sinkronisasi yang telah dibuat tidak akan berfungsi jika tidak diintegrasikan ke dalam alur program utama. Oleh karena itu, bukalah file `src/kernel.c`. Kita akan melindungi area *critical section* dari skenario Producer-Consumer meng-

gunakan instrumen yang baru saja diselesaikan.

1. Cari fungsi `producer(void *aux)`. Hapus tanda komentar (uncomment) pada blok TODO yang memanggil `sema` dan `lock` sehingga kodenya menjadi seperti ini:

```
static void producer(void *aux) {
    (void)aux;
    int item = 0;
    while (1) {
        /* 1. TUNGGU SLOT KOSONG & KUNCI GEMBOK */
        sema_down(&sema_empty);
        lock_acquire(&buf_lock);

        /* --- CRITICAL SECTION --- */
        shared_buffer = item;
        thread_yield(); // Jebakan ini sekarang AMAN karena dilindungi lock
        vga_set_color(VGA_COLOR_BRIGHT_GREEN);
        vga_puts("[Producer] menulis item ");
        vga_putint(item++);
        vga_putchar('\n');
        /* --- END CRITICAL SECTION --- */

        /* 2. BUKA GEMBOK & BERI SINYAL ADA BARANG */
        lock_release(&buf_lock);
        sema_up(&sema_full);

        thread_yield();
    }
}

```

Kode 2.6: Melindungi Critical Section pada Producer

2. Lakukan hal yang sama pada fungsi `consumer(void *aux)`.

```
static void consumer(void *aux) {
```

```
    (void)aux;
    while (1) {
        /* 1. TUNGGU BARANG ADA & KUNCI GEMBOK */
        sema_down(&sema_full);
        lock_acquire(&buf_lock);

        /* --- CRITICAL SECTION --- */
        vga_set_color(VGA_COLOR_YELLOW);
        vga_puts("[Consumer] membaca item ");
        thread_yield(); // Jebakan ini sekarang AMAN karena dilindungi lock
        vga_putint(shared_buffer);
        vga_putchar('\n');
        /* --- END CRITICAL SECTION --- */

        /* 2. BUKA GEMBOK & BERI SINYAL SLOT KOSONG */
        lock_release(&buf_lock);
        sema_up(&sema_empty);

        thread_yield();
    }
}

```

Kode 2.7: Melindungi Critical Section pada Consumer

3. **Simpan** file `kernel.c`.

2.4.5 Sesi 4: Kompilasi dan Verifikasi Sistem OS

1. Lakukan kompilasi sistem operasi Anda dengan *flag* sinkronisasi.

```
# Pastikan telah menjalankan make clean sebelumnya
make DEMO=-DSYNCHRONIZATION
make run

```

Kode 2.8: Compile dan jalankan dengan sinkronisasi penuh

2. **Verifikasi Hasil:** Amati keluaran (*output*) di layar QEMU.
   - **Sebelumnya (Modul 9):** Teks bertumpuk dan *error* karena interupsi `thread_yield()`.
   - **Sekarang (Modul 10):** Setiap `"[Producer] menulis item N"` selalu diikuti secara rapi oleh `"[Consumer] membaca item N"`.

   Meskipun CPU dipaksa berpindah *thread* di tengah jalan oleh jebakan `thread_yield()`, sistem operasi Anda kini sudah memiliki pengamanan memori.
3. Ambil tangkapan layar (*screenshot*) hasil eksekusi untuk membuktikan bahwa implementasi sinkronisasi telah berjalan sesuai ekspektasi.

```
[Producer] menulis item 7235
[Consumer] membaca item 7235
[Producer] menulis item 7236
[Consumer] membaca item 7236
[Producer] menulis item 7237
[Consumer] membaca item 7237
[Producer] menulis item 7238
[Consumer] membaca item 7238
[Producer] menulis item 7239
[Producer] menulis item 7239
[Consumer] membaca item 7239
[Producer] menulis item 7240
[Consumer] membaca item 7240
[Producer] menulis item 7241
[Consumer] membaca item 7241
[Producer] menulis item 7242
[Consumer] membaca item 7242
[Producer] menulis item 7243
[Consumer] membaca item 7243
[Producer] menulis item 7244
[Consumer] membaca item 7244
[Producer] menulis item 7245
[Consumer] membaca item 7245
[Producer] menulis item 7246
[Consumer] membaca item 7246
```

Gambar 2.1: Keluaran (*output*) QEMU setelah implementasi sinkronisasi penuh — Producer dan Consumer bergantian secara rapi, Mutex berhasil mengatasi jebakan Preemption.

Bibliography

[1] Race condition. Section: Operating Systems. [Online]. Available: https://www.geeksforgeeks.org/operating-systems/race-condition-in-operating-systems/

[2] Concurrency in operating system. Section: Operating Systems. [Online]. Available: https://www.geeksforgeeks.org/operating-systems/concurrency-in-operating-system/